

Appendix D: VGG Evidence

In [1]: `!nvidia-smi`

```
Fri Mar 14 01:42:53 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf           Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                       |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off | 00000000:00:04.0 Off |
0|
| N/A    32C    P0              42W / 400W |      0MiB / 40960MiB |      0%
Default |
|              |                       |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |                    Us
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import VGG16 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow import keras # Access to stuff for model
from keras.callbacks import ReduceLROnPlateau # -----
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis o
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Load data

```

In [4]: # Load CIFAR-100 dataset
(X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

# Split (8000) of training data into temporary set
X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_size=
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp, y_te
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5, s

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a 70/
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}")

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 3s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

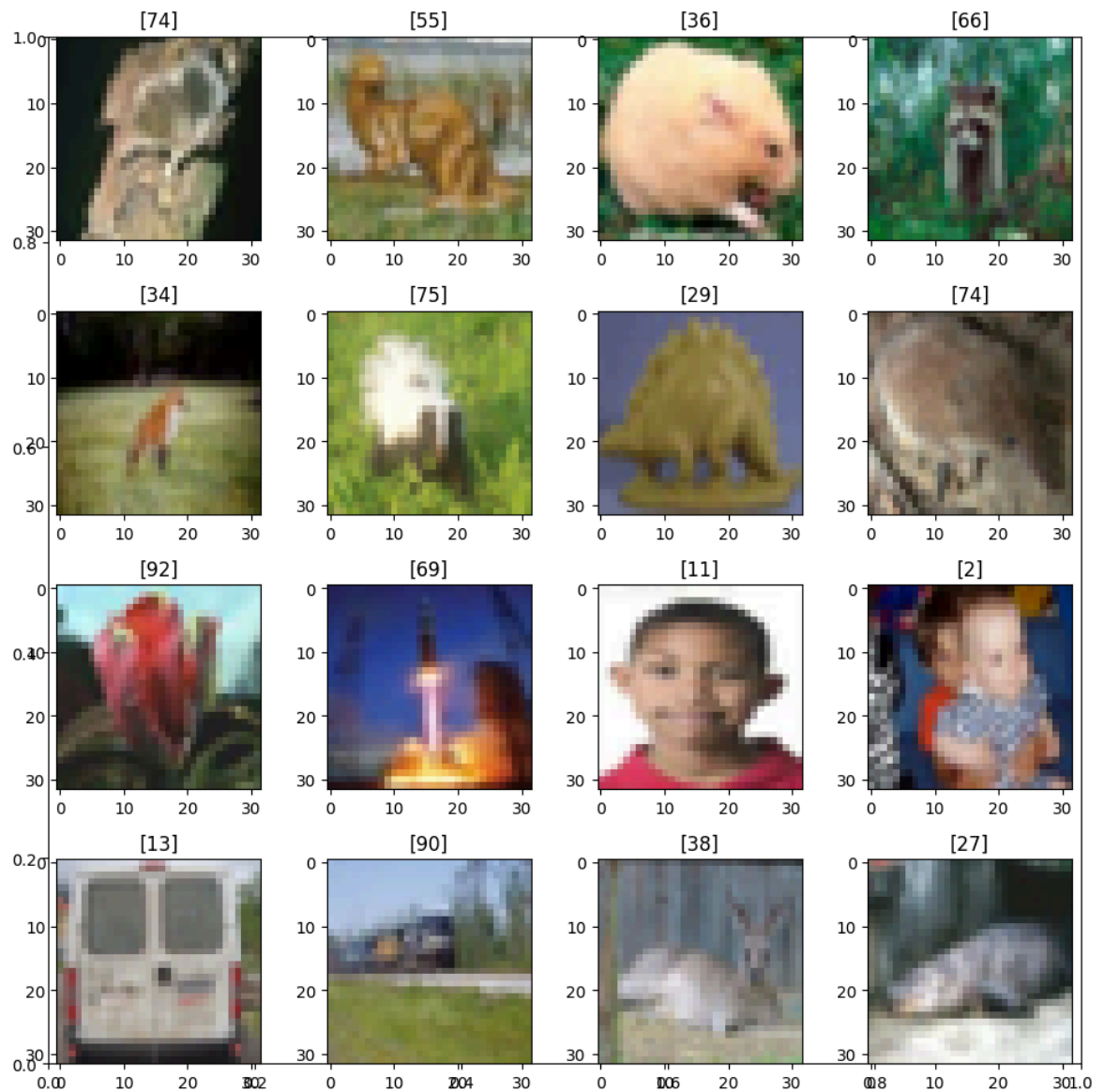
y_test.shape: (9000, 1)

```
In [5]: idxs_disp = []
def display_imgs(imgs, labels, norm_version):
    plt.subplots(figsize=(10,10))
    if norm_version == 0:
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            idxs_disp.append(k)
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.title(labels[k])
            plt.imshow(imgs[k])
    else:
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = idxs_disp[i]
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.title(labels[k])
            plt.imshow(imgs[k])
    plt.tight_layout()
    plt.show()

display_imgs(X_train, y_train, 0)
```

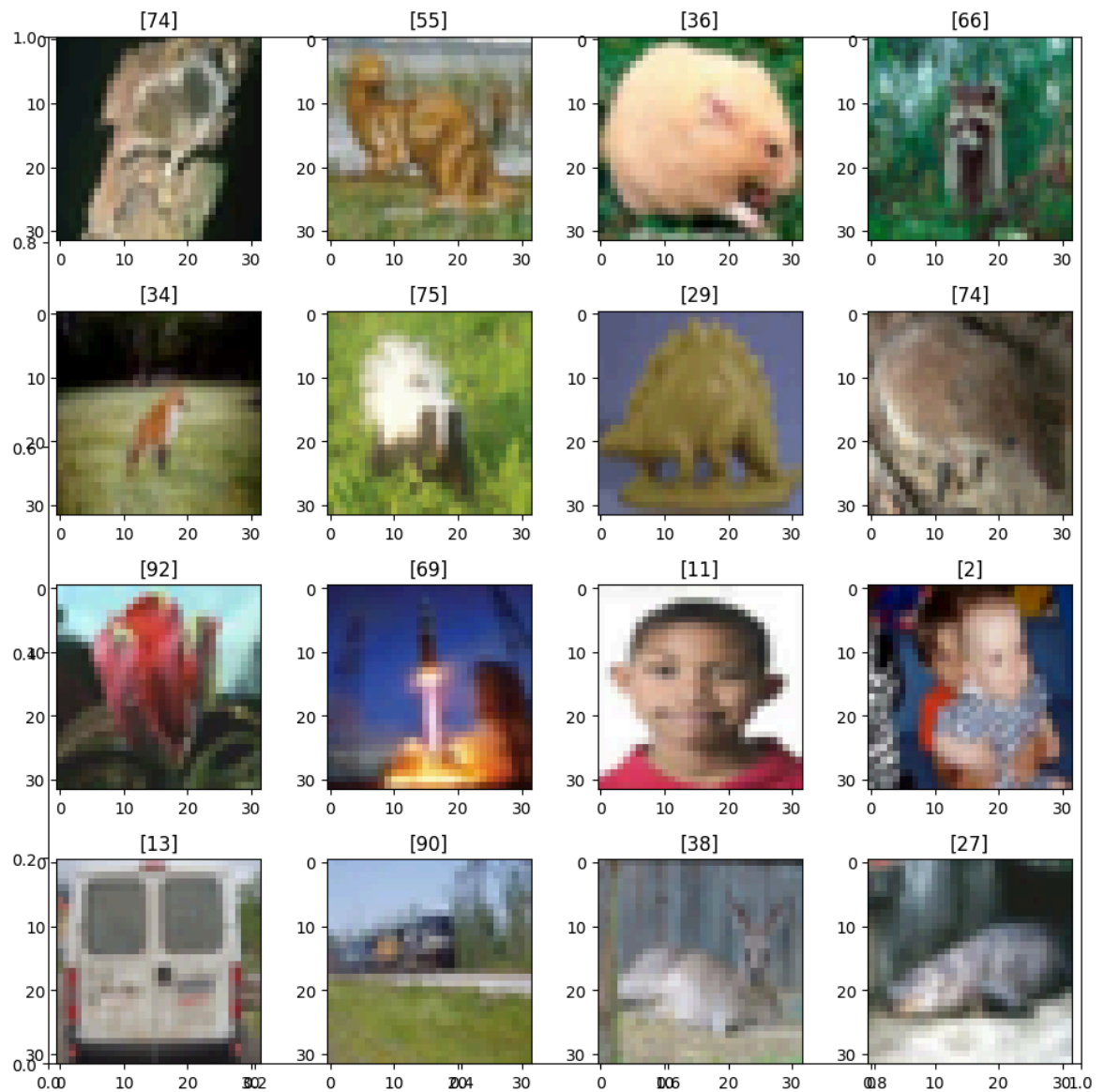
```
labels[11985].shape: (1,)
```

```
imgs[11985].shape: (32, 32, 3)
```



```
In [6]: # Normalise images (scale to range [0, 1]) - Improves convergence speed & accuracy
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train, 1)
```

```
labels[11985].shape: (1,)
imgs[11985].shape: (32, 32, 3)
```



```
In [7]: #train_dataset = tf.data.Dataset.from_tensor_slices((X_train, y_train)).shuffle(
#val_dataset = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(64)
#test_dataset = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(64)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'flatf
'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', 'bowl
'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock', 'c
'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardrobe'
'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wolf',
'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea', 'ca
'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 'skunk
'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile', 'di
'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple', 'c
'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mower',
'tractor']

# Create TensorFlow datasets

batch_size = 64
train_dataset = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                tf.squeeze(tf.one_hot(y, depth=100, dtype=tf
                .batch(batch_size)))
```

```

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))
                                   ).batch(batch_size))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32))
                                   ).batch(batch_size))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3)
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

```

Training dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

```

Validation dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

```

Testing dataset:

```

<_BatchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)

```

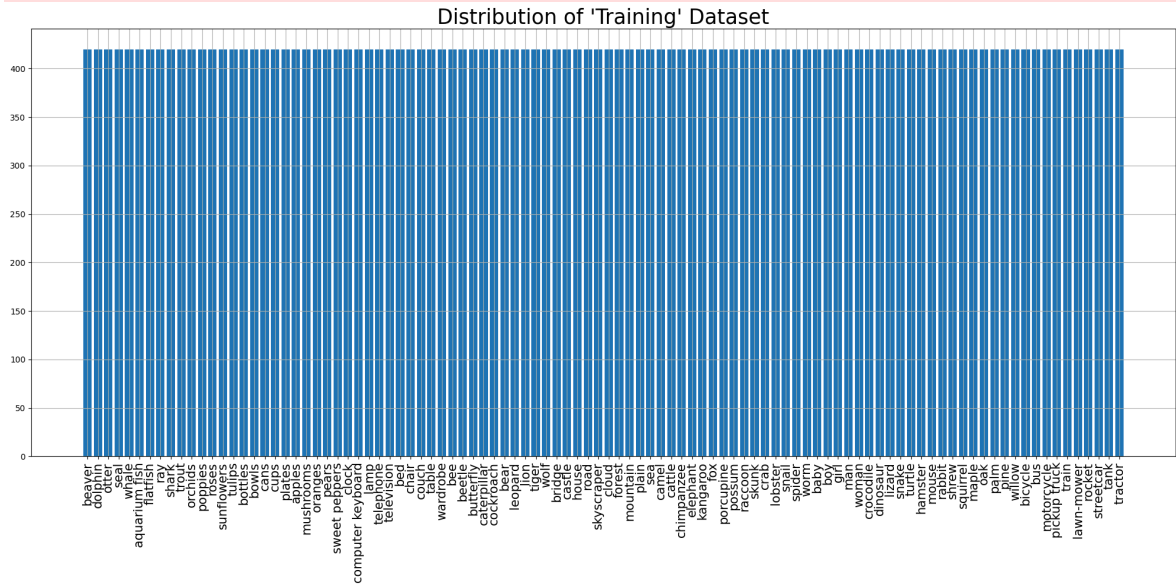
```

In [8]: def class_distrib(y, labels_names, dataset_name):
        counts = pd.DataFrame(data=y).value_counts().sort_index()
        #print(f"counts:\n{counts}")
        fig, ax = plt.subplots(figsize=(20,10))
        ax.bar(labels_names, counts)
        ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
        plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
        plt.grid()
        plt.tight_layout()
        plt.show()

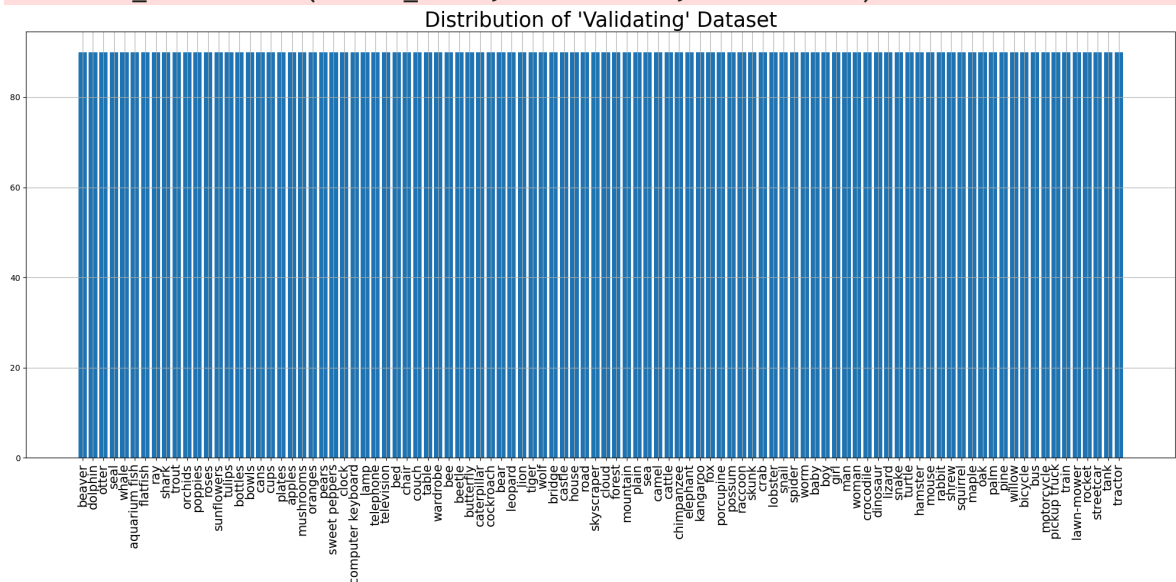
        class_distrib(y_train, labels_names, "Training")
        class_distrib(y_val, labels_names, "Validating")
        class_distrib(y_test, labels_names, "Testing")

```

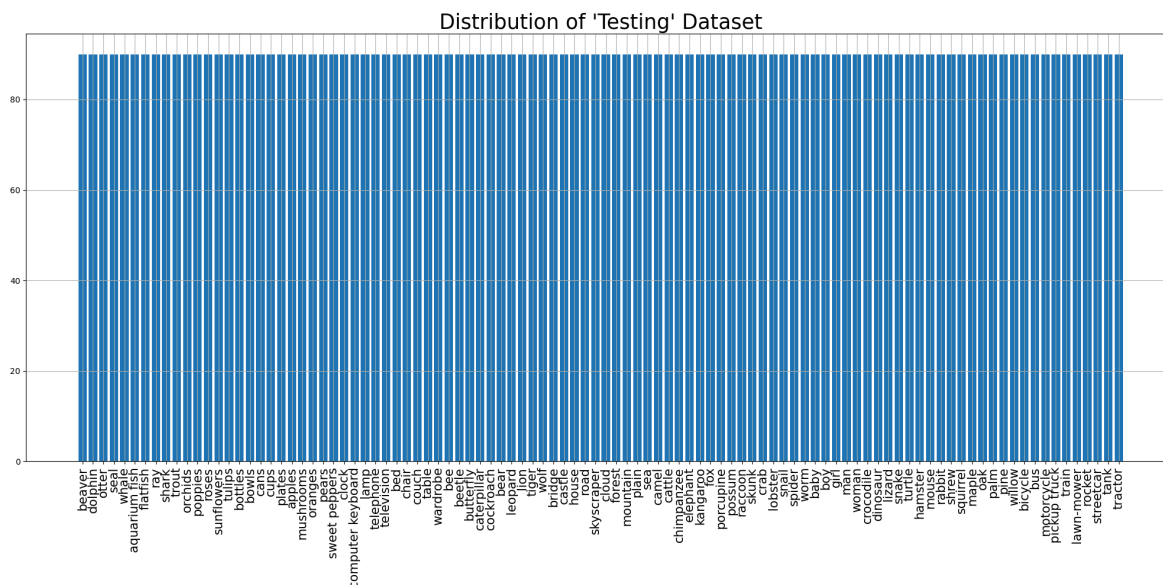
```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-8-b584abab378c>:6: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Define Model

```
In [10]: def build_vgg16():
    model = models.Sequential([
        # First block
        layers.Conv2D(64, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), strides=2),

        # Second block
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), strides=2),

        # Third block
        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), strides=2),

        # Fourth block
        layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), strides=2),

        # Fifth block
        layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
        layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), strides=2),

        # Flatten the output and add fully connected layers
        layers.Flatten(),
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
    ])

    model.compile(optimizer=optimizers.Adam(learning_rate=1e-3),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return model
```



```
# Build the VGG16 model from scratch
model = build_vgg16()
model.summary()
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape
conv2d (Conv2D)	(None, 224, 224, 64)
conv2d_1 (Conv2D)	(None, 224, 224, 64)
max_pooling2d (MaxPooling2D)	(None, 112, 112, 64)
conv2d_2 (Conv2D)	(None, 112, 112, 128)
conv2d_3 (Conv2D)	(None, 112, 112, 128)
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 128)
conv2d_4 (Conv2D)	(None, 56, 56, 256)
conv2d_5 (Conv2D)	(None, 56, 56, 256)
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 256)
conv2d_6 (Conv2D)	(None, 28, 28, 512)
conv2d_7 (Conv2D)	(None, 28, 28, 512)
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 512)
conv2d_8 (Conv2D)	(None, 14, 14, 512)
conv2d_9 (Conv2D)	(None, 14, 14, 512)
max_pooling2d_4 (MaxPooling2D)	(None, 7, 7, 512)
flatten (Flatten)	(None, 25088)
dense (Dense)	(None, 512)
dropout (Dropout)	(None, 512)
dense_1 (Dense)	(None, 100)

Total params: 22,301,860 (85.07 MB)

Trainable params: 22,301,860 (85.07 MB)

Non-trainable params: 0 (0.00 B)

Train Model & Track Training/Validation Error

```
In [11]: # reduceLR = ReduceLROnPateau(monitor='val_accuracy', factor=0.1, patience=3, mi

# Train model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=20, verbo
```

Epoch 1/20

657/657  **102s** 114ms/step - accuracy: 0.0091 - loss: 4.6322 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 2/20

657/657  **58s** 88ms/step - accuracy: 0.0089 - loss: 4.6056 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 3/20

657/657  **58s** 88ms/step - accuracy: 0.0088 - loss: 4.6057 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 4/20

657/657  **58s** 88ms/step - accuracy: 0.0076 - loss: 4.6058 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 5/20

657/657  **58s** 88ms/step - accuracy: 0.0082 - loss: 4.6058 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 6/20

657/657  **58s** 88ms/step - accuracy: 0.0083 - loss: 4.6058 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 7/20

657/657  **58s** 88ms/step - accuracy: 0.0084 - loss: 4.6058 - val_accuracy: 0.0100 - val_loss: 4.6052

Epoch 8/20

359/657  **24s** 83ms/step - accuracy: 0.0083 - loss: 4.6058

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
<ipython-input-11-43f65880634d> in <cell line: 0>()
      2
      3 # Train model
----> 4 history = model.fit(train_dataset, validation_data=val_dataset, epochs=2
0, verbose=1)

/usr/local/lib/python3.11/dist-packages/keras/src/utils/traceback_utils.py in error_handler(*args, **kwargs)
    115         filtered_tb = None
    116         try:
--> 117             return fn(*args, **kwargs)
    118         except Exception as e:
    119             filtered_tb = _process_traceback_frames(e.__traceback__)

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py in fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch, validation_steps, validation_batch_size, validation_freq)
    369         for step, iterator in epoch_iterator:
    370             callbacks.on_train_batch_begin(step)
--> 371             logs = self.train_function(iterator)
    372             callbacks.on_train_batch_end(step, logs)
    373             if self.stop_training:

/usr/local/lib/python3.11/dist-packages/keras/src/backend/tensorflow/trainer.py in function(iterator)
    218         ):
    219             opt_outputs = multi_step_on_iterator(iterator)
--> 220             if not opt_outputs.has_value():
    221                 raise StopIteration
    222             return opt_outputs.get_value()

/usr/local/lib/python3.11/dist-packages/tensorflow/python/data/ops/optional_ops.py in has_value(self, name)
    174     def has_value(self, name=None):
    175         with ops.colocate_with(self._variant_tensor):
--> 176             return gen_optional_ops.optional_has_value(
    177                 self._variant_tensor, name=name
    178             )

/usr/local/lib/python3.11/dist-packages/tensorflow/python/ops/gen_optional_ops.py in optional_has_value(optional, name)
    170     if tld.is_eager:
    171         try:
--> 172             _result = pywrap_tfe.TFE_Py_FastPathExecute(
    173                 _ctx, "OptionalHasValue", name, optional)
    174             return _result

KeyboardInterrupt:

```